National Economics University
Faculty of Mathematical Economics

Database System Design, Implementation And Management Course – Advanced SQL

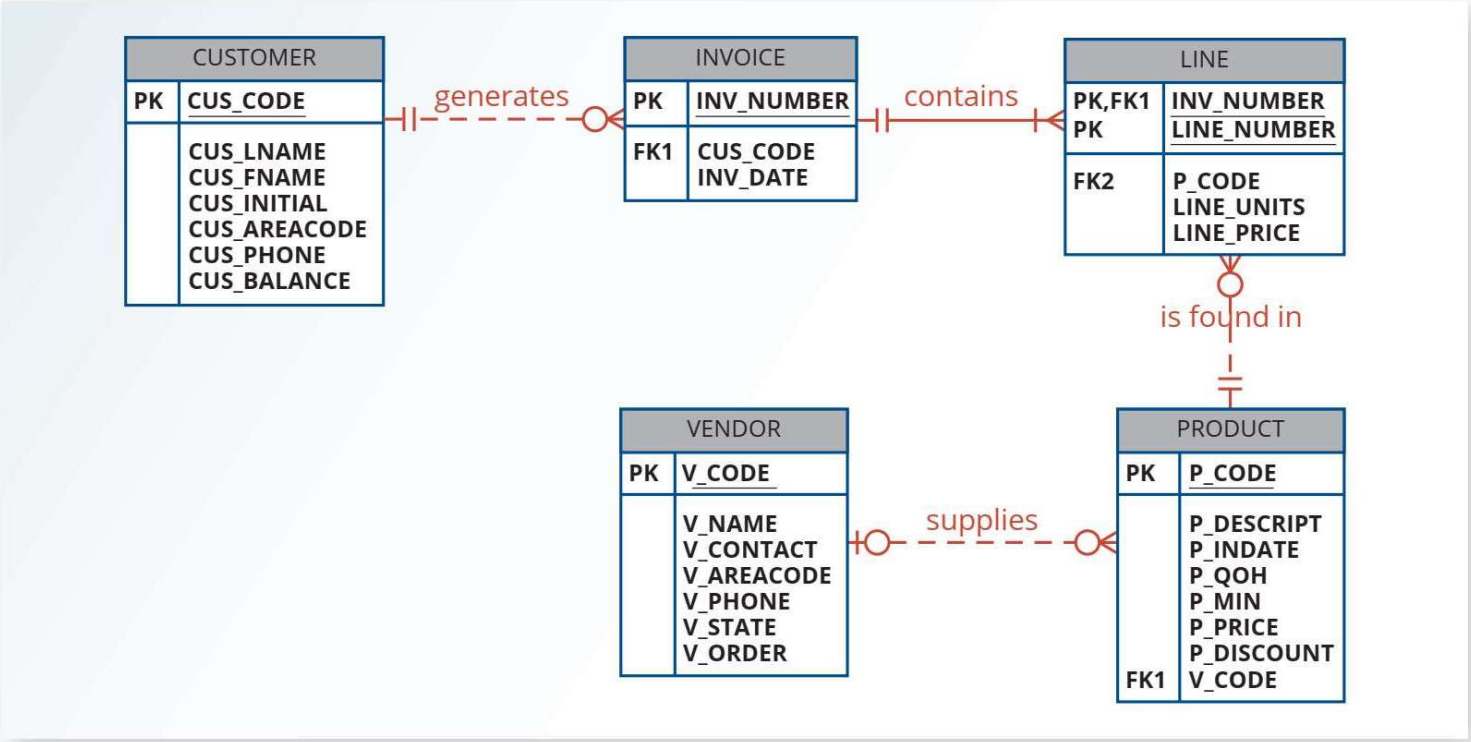
- Instructor: Duc Minh Vu (MFE)
- Email: minhvd [at] neu.edu.vn

Learning Objectives

- 8-1 Use SQL to create a table manually
- 8-2 Use SQL to create a copy of a table using a subquery
- 8-3 Manipulate the structure of existing tables to add, modify, and remove columns and constraints
- 8-4 Use SQL to do data manipulation (insert, update, and deleterows of data)
- 8-5 Use SQL to create database views, including updatable views
- 8-6 Use procedural SQL to create triggers, stored procedures, and procedural SQL functions
- 8-7 Create embedded SQL

8-1 Data Definition Commands

Figure 8.1 Database Model



TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

8-1b Creating the Database

- If you are using an enterprise RDBMS, you must be authenticated by the RDBMS before
  you can start creating tables.
- **Authentication** The process through which a DBMS verifies that only registered users can access the database.
- To be authenticated, you must log on to the RDBMS using a user ID and a password created by the database administrator.
- In an enterprise RDBMS, every user ID is associated with a database schema.

8-1c The Database Schema

- a **schema** is a logical group of database objects—such as tables and indexes—that are related to each other
- the schema belongs to a single user or application
- A single database can hold multiple schemas that belong to different users or applications.
- Schemas [illegible] force a first level [illegible] to that user

ANSI SQL standards define a command to create a database schema:

CREATE SCHEMA AUTHORIZATION {creator};

Therefore, if the creator is JONES, the following command is used:

CREATE SCHEMA AUTHORIZATION JONES;

Table 8.1 Some Common SQL Data Types

Data Type	Format	Comments
Numeric	NUMERIC(L,D) or NUMBER(L,D)	The declaration NUMERIC(7,2) or NUMBER (7,2) indicates that numbers will be stored with two decimal places and may be up to seven digits long, including the sign and the decimal place (for example, 12.32 or –134.99).
	INTEGER	May be abbreviated as INT. Integers are (whole) counting numbers, so they cannot be used if you want to store numbers that require decimal places.
	SMALLINT	Like INTEGER but limited to integer values up to six digits. If your integer values are relatively small, use SMALLINT instead of INT.
	DECIMAL(L,D)	Like the NUMERIC specification, but the storage length is a *minimum* specification. That is, greater lengths are acceptable, but smaller ones are not. DECIMAL(9,2), DECIMAL(9), and DECIMAL are all acceptable. In most popular DBMSs, this difference is not implemented so NUMERIC and DECIMAL operate the same.
Character	CHAR(L)	Fixed-length character data for up to 255 characters. If you store strings that are not as long as the CHAR parameter value, the remaining spaces are left unused. Therefore, if you specify CHAR(25), strings such as *Smith* and *Katzenjammer* are each stored as 25 characters. However, a U.S. area code is always three digits long, so CHAR(3) would be appropriate if you wanted to store such codes.
	VARCHAR(L) or VARCHAR2(L)	Variable-length character data. The designation VARCHAR(25) or VARCHAR2(25) will let you store up to 25 characters. However, unlike CHAR, VARCHAR will not leave unused spaces. Oracle automatically converts VARCHAR to VARCHAR2.
Date	DATE	Stores dates in the Julian date format.

8-2 Creating Table Structures

8-2a CREATE TABLE command

- **CREATE TABLE** A SQL command that creates a table's structures using the characteristics and attributes given.

```
CREATE TABLE tablename (
column1        data type      [constraint] [,
column2        data type      [constraint] ] [,
PRIMARY KEY    (column1       [, column2]) ] [,
FOREIGN KEY    (column1       [, column2]) REFERENCES tablename] [,
CONSTRAINT     constraint ]);
```

Note

SQL Syntax

Syntax notation for SQL commands used in this book:

CAPITALS	Required SQL command keywords
italics	A parameter provided by the end user (generally required)
{a | b | ..}	A mandatory parameter; use one option from the list separated by |
[……]	An optional parameter—anything inside square brackets is optional
Tablename	The name of a table
Column	The name of an attribute in a table
data type	A valid data-type definition
constraint	A valid constraint definition
condition	A valid conditional expression (evaluates to true or false)
columnlist	One or more column names or expressions separated by commas
tablelist	One or more table names separated by commas
conditionlist	One or more conditional expressions separated by logical operators
expression	A simple value (such as 76 or Married) or a formula (such as P_PRICE – 10)


```
CREATE TABLE VENDOR (
V_CODE          INTEGER         NOT NULL        UNIQUE,
V_NAME          VARCHAR(35)     NOT NULL,
V_CONTACT       VARCHAR(25)     NOT NULL,
V_AREACODE      CHAR(3)         NOT NULL,
V_PHONE         CHAR(8)         NOT NULL,
V_STATE         CHAR(2)         NOT NULL,
V_ORDER         CHAR(1)         NOT NULL,
PRIMARY KEY (V_CODE));
```

```
CREATE TABLE PRODUCT (
P_CODE          VARCHAR(10)     NOT NULL        UNIQUE,
P_DESCRIPT      VARCHAR(35)     NOT NULL,
P_INDATE        DATE            NOT NULL,
P_QOH           SMALLINT        NOT NULL,
P_MIN           SMALLINT        NOT NULL,
P_PRICE         NUMBER(8,2)     NOT NULL,
P_DISCOUNT      NUMBER(5,2)     NOT NULL,
V_CODE          INTEGER,
PRIMARY KEY (P_CODE),
FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE) ON UPDATE
CASCADE);
```


Note

If you are working with a composite primary key, all of the primary key's attributes are contained within the parentheses and are separated with commas. For example, the LINE table in Figure 8.1 has a primary key that consists of the two attributes INV_NUMBER and LINE_NUMBER. Therefore, you would define the primary key by typing the following:

PRIMARY KEY (INV_NUMBER, LINE_NUMBER),

The order of the primary key components is important because the indexing starts with the first mentioned attribute, then proceeds with the next attribute, and so on. In this example, the line numbers would be ordered within each of the invoice numbers:

INV_NUMBER	LINE_NUMBER
1001	1
1001	2
1002	1
1003	1
1003	2

The ON UPDATE CASCADE specification ensures that if you make a change in any VENDOR's V_CODE that change is automatically applied to all foreign key references throughout the system to ensure that referential integrity is maintained. (Although the ON UPDATE CASCADE clause is part of the ANSI standard, some RDBMSs, such as Oracle, do not support it. If your RDBMS does not support the clause, delete it from the code shown here.)

Note

Note About Referential Constraint Actions

The support for the referential constraint's actions varies from product to product. For example:

- MySQL requires the InnoDB storage engine to enforce referential integrity.
- MS Access, SQL Server, MySQL, and Oracle support ON DELETE CASCADE.
- MS Access, SQL Server, and MySQL support ON UPDATE CASCADE.
- Oracle does not support ON UPDATE CASCADE.
- Oracle and MySQL support SET NULL.
- MS Access and SQL Server do not support SET NULL.
- Refer to your product manuals for additional information on referential constraints.

While MS Access does not support ON DELETE CASCADE or ON UPDATE CASCADE at the SQL command-line level, it does support them through the relationship window interface. In fact, whenever you try to establish a relationship between two tables in Access, the relationship window interface will automatically pop up.

- The CHECK constraint is used to validate data when an attribute value is entered. The CHECK constraint does precisely what its name suggests: it checks to see that a specified condition exists. Examples of such constraints include the following:
  - The minimum order value must be at least 10.
  - The date must be after April 15, 2018.
  - If the CHECK constraint is met for the specified attribute (i.e., the condition is true), the data is accepted for that attribute. If the condition is found to be false, an error message is generated and the data is not accepted.


```
CREATE TABLE CUSTOMER (
CUS_CODE        INTEGER        PRIMARY KEY,
CUS_LNAME       VARCHAR(15)    NOT NULL,
CUS_FNAME       VARCHAR(15)    NOT NULL,
CUS_INITIAL     CHAR(1),
CUS_AREACODE    CHAR(3)        DEFAULT '615'          NOT NULL
                CHECK(CUS_AREACODE IN ('615','713','931')),
CUS_PHONE       CHAR(8)        NOT NULL,
CUS_BALANCE     DECIMAL(9,2)   DEFAULT 0.00,
CONSTRAINT CUS_UI1 UNIQUE (CUS_LNAME, CUS_FNAME));
```

In this case, the CUS_AREACODE attribute is assigned a default value of '615'. Therefore, if a new CUSTOMER table row is added and the end user makes no entry for the area code, the '615' value will be recorded. Also, the CHECK condition restricts the values for the customer's area code to 615, 713, and 931; any other values will be rejected.

The DEFAULT value applies only when new rows are added to a table and then only when no value is entered for the customer's area code. (The default value is not used when an existing row in the table is modified.) In contrast, the CHECK condition is validated whether a customer row is added or *modified*. However, while the CHECK condition may include any valid expression, it applies only to the attributes in the table being checked. If you want to check for conditions that include attributes in other tables, you must use triggers, as discussed later in this chapter. Finally, the last line of the CREATE TABLE command sequence creates a unique index constraint (named CUS_UI1) on the customer's last name and first name. The index will prevent the entry of two customers with the same last name and first name. (This index merely illustrates the process. Clearly, it should be possible to have more than one person named John Smith in the CUSTOMER table.)


```
CREATE TABLE INVOICE (
INV_NUMBER    INTEGER     PRIMARY KEY,
CUS_CODE      INTEGER     NOT NULL REFERENCES CUSTOMER(CUS_CODE),
INV_DATE      DATETIME    DEFAULT NOW() NOT NULL,
CONSTRAINT INV_CK1 CHECK (INV_DATE > '2022-01-01'));
```

- The CUS_CODE attribute definition contains REFERENCES CUSTOMER(CUS_CODE) to indicate that the CUS_CODE is a foreign key. This is another way to define a foreign key.
- The DEFAULT constraint uses the NOW() function. This function always returns today's date.
- The invoice date (INV_DATE) attribute is automatically given today's date (returned by NOW()) when a new row is added if no value is given for the attribute.
- A CHECK constraint is used to validate that the invoice date is greater than "January 1, 2022."


```
CREATE TABLE LINE (
INV_NUMBER      INTEGER         NOT NULL,
LINE_NUMBER     DECIMAL(2,0)    NOT NULL,
P_CODE          VARCHAR(10)     NOT NULL,
LINE_UNITS      DECIMAL(9,2)    DEFAULT 0.00    NOT NULL,
LINE_PRICE      DECIMAL(9,2)    DEFAULT 0.00    NOT NULL,
PRIMARY KEY (INV_NUMBER, LINE_NUMBER),
FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE (INV_NUMBER) ON
DELETE CASCADE,
FOREIGN KEY (P_CODE) REFERENCES PRODUCT(P_CODE),
CONSTRAINT LINE_UI1 UNIQUE(INV_NUMBER, P_CODE));
```

In the creation of the LINE table, note that a UNIQUE constraint is added to prevent the duplication of an invoice line. A UNIQUE constraint is enforced through the creation of a unique index. Also note that the ON DELETE CASCADE foreign key enforces referential integrity. The use of ON DELETE CASCADE is recommended for weak entities to ensure that the deletion of a row in the strong entity automatically triggers the deletion of the corresponding rows in the dependent weak entity. In that case, the deletion of an INVOICE row will automatically delete all of the LINE rows related to the invoice. In the following section, you will learn more about indexes and how to use SQL commands to create them.

8-2c Creating a Table with a SELECT Statement

- *SQL provides a way to rapidly create a new table based on selected columns and rows of an existing table using a subquery.*

```
CREATE TABLE [IF NOT EXISTS] PART AS
SELECT    P_CODE AS PART_CODE, P_DESCRIPT AS PART_DESCRIPT, P_PRICE
          AS PART_PRICE, V_CODE
FROM      PRODUCT;
```

8-2d SQL Indexes

- **CREATE INDEX** A SQL command that creates indexes on the basis of a selected attribute or attributes
- indexes can be used to improve the efficiency of searches and to avoid duplicate column values
- when you declare a primary key, the DBMS automatically creates a unique index

CREATE [UNIQUE]INDEX *indexname* ON *tablename*(*column1* [, *column2*])

For example, based on the attribute P_INDATE stored in the PRODUCT table, the following command creates an index named P_INDATEX:

CREATE INDEX P_INDATEX ON PRODUCT(P_INDATE);

8-2d SQL Indexes

- **DROP INDEX** A SQL command used to delete database objects such as tables, views, indexes, and users.

To delete an index, use the **DROP INDEX** command:

DROP INDEX *indexname*

While the ANSI standard for SQL only requires the index name, some DBMSs such as MySQL require specifying the table name as well. For example, if you want to eliminate the PROD_PRICEX index in MySQL, type:

DROP INDEX PROD_PRICEX ON PRODUCT;

8-3 Altering Table Structures

8-3 Altering Table Structures

- **ALTER TABLE** The SQL command used to make changes to table structure. When the command is followed by a keyword (ADD, ALTER, or MODIFY), it adds a column or changes column

ALTER TABLE *tablename*
{ADD | MODIFY | ALTER} [COLUMN] (*columnname datatype* [{ADD | MODIFY | ALTER} *columnname datatype*]);

ALTER TABLE *tablename*
ADD *constraint* [ADD *constraint*];

ALTER TABLE *tablename*
DROP {PRIMARY KEY | COLUMN *columnname* | CONSTRAINT *constraintname* };

```
ALTER TABLE PRODUCT
    MODIFY V_CODE CHAR(5);
```

```
ALTER TABLE PRODUCT
    MODIFY P_PRICE DECIMAL(9,2);
```

```
ALTER TABLE PRODUCT
    ADD (P_SALECODE CHAR(1));
```

```
ALTER TABLE    PART
ADD            PRIMARY KEY (PART_CODE);
```

```
ALTER TABLE    PART
ADD            FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE);
```

```
ALTER TABLE    PART
ADD            CHECK (PART_PRICE >= 0);
```



```
ALTER TABLE    PART
ADD            PRIMARY KEY (PART_CODE),
ADD            FOREIGN KEY (V_CODE) REFERENCES VENDOR (V_CODE),
ADD            CHECK (PART_PRICE >= 0);
```

```
ALTER TABLE    LINE
ADD            PRIMARY KEY (INV_NUMBER, LINE_NUMBER)
ADD            FOREIGN KEY (INV_NUMBER) REFERENCES INVOICE
               (INV_NUMBER)
ADD            FOREIGN KEY (P_CODE) REFERENCES PRODUCT (P_CODE);
```

```
ALTER TABLE VENDOR
    DROP COLUMN V_ORDER;
```

```
DROP TABLE PART;
```

8-4 Data Manipulation Commands

8-4a Adding Table Rows

- INSERT INTO *tablename* VALUES (*value1, value2, …, valuen*)

```
INSERT INTO VENDOR
    VALUES (21225, 'Bryson, Inc.', 'Smithson', '615', '223-3234', 'TN', 'Y');
INSERT INTO VENDOR
    VALUES (21226, 'Superloo, Inc.', 'Flushing', '904', '215-8995', 'FL', 'N');
```

```
INSERT INTO PRODUCT(P_CODE, P_DESCRIPT) VALUES ('BRT-345', 'Titanium drill bit');
```

8-4b Inserting Table Rows with a SELECT Subquery

```
INSERT INTO     target_tablename[(target_columnlist)]
SELECT          source_columnlist
FROM            source_tablename;
```

```
INSERT INTO PART    (PART_CODE, PART_DESCRIPT, PART_PRICE, V_CODE)
SELECT              P_CODE, P_DESCRIPT, P_PRICE, V_CODE
FROM PRODUCT;
```


8-4c Saving Table Changes

- **COMMIT** The SQL command that terminates a transaction by permanently saving data changes to a database.
- The COMMIT command permanently saves all changes—such as rows added, attributes modified, and rows deleted—made to any table in the database
- COMMIT [WORK | TRANSACTION]

8-4d Updating Table Rows

- **UPDATE** A SQL command that allows attribute values to be changed in one or more ro

```
UPDATE      tablename
SET         columnname = expression [, columnname = expression]
[WHERE      conditionlist];
```

```
UPDATE      PRODUCT
SET         P_INDATE = '2022-01-18'
WHERE       P_CODE = '13-Q2/P2';
```

```
UPDATE      PRODUCT
SET         P_INDATE = '2022-01-18', P_PRICE = 17.99, P_MIN = 10
WHERE       P_CODE = '13-Q2/P2';
```


TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

8-4e Deleting Table Rows

- **DELETE** A SQL command that allows data rows to be deleted from a table.

```
DELETE FROM     tablename
[WHERE          conditionlist];
```

```
DELETE FROM     PRODUCT
WHERE           P_CODE = 'BRT-345';
```

8-4f Restoring Table Contents

- **ROLLBACK** A SQL command that restores the database table contents to the condition that existed after the last COMMIT statement.
  - ROLLBACK;
- If you have not yet used the COMMIT command to store the changes permanently in the database, you can restore the database to its previous condition with the **ROLLBACK** command.
- COMMIT and ROLLBACK work only with data manipulation commands that add, modify, or delete table rows

COMMIT and ROLLBACK work only with data manipulation commands that add, modify, or delete table rows. For example, assume that you perform these actions:

1. CREATE a table called SALES.
2. INSERT 10 rows in the SALES table.
3. UPDATE two rows in the SALES table.
4. Execute the ROLLBACK command.

Will the SALES table be removed by the ROLLBACK command? No, the ROLLBACK command will undo *only* the results of the INSERT and UPDATE commands. All data definition commands (CREATE TABLE) are automatically committed to the data dictionary and cannot be rolled back. The COMMIT and ROLLBACK commands are examined in greater detail in Chapter 10.

8-5 Virtual Tables: Creating a View

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

8-5 Virtual Tables: Creating a View

- **View** A virtual table based on a SELECT query that is saved as an object in the database.
- **base table** The table on which a view is based.
- **CREATE VIEW** A SQL command that creates a logical, "virtual" table. The view can be treated as a real table.
- CREATE VIEW *viewname* [*(column list)*] AS SELECT *query*

```
CREATE VIEW PROD_STATS AS
SELECT      V_CODE, SUM(P_QOH*P_PRICE) AS TOTCOST, MAX(P_QOH)
            AS MAXQTY, MIN(P_QOH) AS MINQTY, AVG(P_QOH) AS AVGQTY
FROM        PRODUCT
GROUP BY    V_CODE;
```


Figure 8.5 Creating a Virtual Table with the Create View Command


```
CREATE VIEW PRICEGT50 AS
    SELECT P_DESCRIPT, P_QOH, P_PRICE
    FROM PRODUCT
    WHERE P_PRICE > 50;

SELECT * FROM PRICEGT50;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

P_DESCRIPT	P_QOH	P_PRICE
Power painter, 15 psi., 3-nozzle	8	109.99
B&D jigsaw, 12-in. blade	8	109.92
B&D jigsaw, 8-in. blade	6	99.87
Hicut chain saw, 16 in.	11	256.99
Steel matting, 4'x8'x1/6", .5" mesh	18	119.95

8-6 Auto Increment, Identity, and Sequences

8-7 Procedural SQL

8-7 Procedural SQL

- **persistent storage module (PSM)** A block of code with standard SQL statements and procedural extensions that is stored and executed at the DBMS server.
- The PSM represents business logic that can be encapsulated, stored, and shared among multiple database users.
- A PSM lets an administrator assign specific access rights to a stored module to ensure that only authorized users can use it.
- **Procedural SQL** An extension of the SQL programming language, such as PL/SQL or TSQL, that add procedural programming capabilities, such as variable and logical flow control, to SQL and is designed to run inside the database.

8-7a Stored Procedures

- **stored procedure** (1) A named collection of procedural and SQL statements
- They can be used to encapsulate and represent business transactions
- Stored procedures substantially reduce network traffic and increase performance.
- Stored procedures help reduce code duplication by means of code isolation and code sharing (creating unique modules that are called by application programs)

```
CREATE PROCEDURE procedure_name ([IN | OUT | INOUT] argument data-type, …)
BEGIN
        [DECLARE variable_name data type [DEFAULT initial_value] ]
        [BEGIN]
                Procedural SQL and SQL statements;
        …
        [END;]
END;
```


TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

- *argument* specifies the parameters that are passed to the stored procedure. A stored procedure could have zero or more arguments or parameters.
- *IN | OUT | INOUT* indicates whether the parameter is for input, output, or both.
- *data-type* is one of the procedural SQL data types used in the RDBMS. The data types normally match those used in the RDBMS table creation statement.
- The body of the stored procedure must be inside a BEGIN…END; block. Within that block can be one or more DECLARE statements and other blocks of executable code.
- Variables can be declared in a DECLARE statement. You must specify the variable name, its data type, and (optionally) an initial value.

Figure 8.9 Block Program with Insert

```
delimiter $$
DROP PROCEDURE IF EXISTS TEMP_PROC$$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    INSERT INTO VENDOR VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
END;
$$
delimiter ;

call temp_proc()
```


Figure 8.10 Output to the Screen

```
delimiter $$
DROP PROCEDURE IF EXISTS TEMP_PROC$$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    INSERT INTO VENDOR VALUES (25678, 'Microsoft Corp.', 'Bill Gates', '765', '546-8484', 'WA', 'N');
    SELECT 'New Vendor Added!' as 'Result';
END;
$$
delimiter ;

call temp_proc()
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
New Vendor Added!

8-7b Working with Variables

Figure 8.11 Variable Manipulation


```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM1 NUMERIC(1) DEFAULT 5;
    DECLARE MYNUM2 NUMERIC(1);
    DECLARE TOTAL INT;
    DECLARE MESSAGE VARCHAR(15) DEFAULT 'The total is ';

    SET MYNUM2 = 6;
    SET TOTAL = MYNUM1 * MYNUM2;
    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
END;
$$
delimiter ;

call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
The total is 30

Figure 8.12 Conditional Execution

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM1 INT DEFAULT 5;
    DECLARE MYNUM2 INT DEFAULT 60;
    DECLARE TOTAL INT;
    DECLARE MESSAGE VARCHAR(50);

    SET TOTAL = MYNUM1 * MYNUM2;
    IF TOTAL < 100 THEN
        SET MESSAGE = 'The total is small: ';
    ELSE
        SET MESSAGE = 'The total is large: ';
    END IF;
    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
END;
$$
delimiter ;

call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
The total is large: 300

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

Figure 8.13 ELSEIF for Complex IF Structure

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM1 INT DEFAULT 50;
    DECLARE MYNUM2 INT DEFAULT 10;
    DECLARE TOTAL INT;
    DECLARE MESSAGE VARCHAR(50);
    SET TOTAL = MYNUM1 * MYNUM2;
    IF TOTAL < 100 THEN
        SET MESSAGE = 'The total is small: ';
    ELSEIF TOTAL >= 100 AND TOTAL <= 500 THEN
        SET MESSAGE = 'The total is medium: ';
    ELSEIF TOTAL BETWEEN 500 AND 1000 THEN
        SET MESSAGE = 'The total is big: ';
    ELSE
        SET MESSAGE = 'The total is huge: ';
    END IF;
    SELECT CONCAT(MESSAGE, TOTAL) AS 'Result';
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Result
The total is medium: 500

Figure 8.14 A Simple Loop

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM INT DEFAULT 1;
    DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
    COUNTER: LOOP
    SET MYNUM = MYNUM + 1;
    IF MYNUM > 20 THEN
        LEAVE COUNTER;
    END IF;
    SET RESULT = CONCAT(RESULT, ', ', MYNUM);
    END LOOP;
    SELECT RESULT;
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RESULT
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

Figure 8.15 Implementation of a WHILE Loop

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM INT DEFAULT 1;
    DECLARE RESULT VARCHAR(100) DEFAULT MYNUM;
    WHILE MYNUM < 20 DO
        SET MYNUM = MYNUM + 1;
        SET RESULT = CONCAT(RESULT, ', ', MYNUM);
    END WHILE;
    SELECT RESULT;
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RESULT
1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

Figure 8.17 Temporary Table for Outputs

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE MYNUM INT DEFAULT 1;
    CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
    WHILE MYNUM <= 5 DO
        INSERT INTO MYTAB VALUES (CONCAT('The current value is ', MYNUM));
        SET MYNUM = MYNUM + 1;
    END WHILE;
    SELECT * FROM MYTAB;
    DROP TEMPORARY TABLE MYTAB;
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

RESULT
The current value is 1
The current value is 2
The current value is 3
The current value is 4
The current value is 5

Figure 8.19 An Explicit Cursor Example

```
DROP PROCEDURE IF EXISTS temp_proc;
delimiter $$
CREATE PROCEDURE TEMP_PROC()
BEGIN
    DECLARE W_P_CODE VARCHAR(15);
    DECLARE W_P_DESCRIPT VARCHAR(50);
    DECLARE W_TOT INT DEFAULT 0;
    DECLARE NOTFOUND INT DEFAULT 0;
    DECLARE PROD_CURSOR CURSOR FOR SELECT P_CODE, P_DESCRIPT
                                   FROM PRODUCT
                                   WHERE P_QOH > (SELECT AVG(P_QOH) FROM PRODUCT);
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET NOTFOUND = 1;
    DROP TEMPORARY TABLE IF EXISTS MYTAB;
    CREATE TEMPORARY TABLE MYTAB(RESULT VARCHAR(100));
    INSERT INTO MYTAB VALUES ('==========');
    OPEN PROD_CURSOR;
    GET_PRODUCTS: LOOP
        FETCH PROD_CURSOR INTO W_P_CODE, W_P_DESCRIPT;
        IF NOTFOUND = 1 THEN
            LEAVE GET_PRODUCTS;
        END IF;
        INSERT INTO MYTAB VALUES (CONCAT(W_P_CODE, ' -> ', W_P_DESCRIPT));
        SET W_TOT = W_TOT + 1;
    END LOOP;
    CLOSE PROD_CURSOR;
    INSERT INTO MYTAB VALUES ('==========');
    INSERT INTO MYTAB VALUES (CONCAT('TOTAL PRODUCTS PROCESSED: ', W_TOT));
    INSERT INTO MYTAB VALUES ('--- END OF REPORT ---');
    SELECT RESULT AS "PRODUCTS WITH P_QOH > AVG(P_QOH)" FROM MYTAB;
    DROP TEMPORARY TABLE MYTAB;
END;
$$
delimiter ;
call temp_proc();
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

PRODUCTS WITH P_QOH > AVG(P_QOH)
==========
PVC23DRT -> PVC pipe, 3.5-in., 8-ft
SM-18277 -> 1.25-in. metal screw, 25
SW-23116 -> 2.5-in. wd. screw, 50
==========
TOTAL PRODUCTS PROCESSED: 3
--- END OF REPORT ---

Figure 8.21 Second Version of the PRC_PROD_DISCOUNT Stored Procedure

```
DROP PROCEDURE IF EXISTS prc_prod_discount;
delimiter $$
CREATE PROCEDURE PRC_PROD_DISCOUNT(IN WPI DECIMAL(2,2))
BEGIN
    IF WPI <=0 OR WPI >= 1 THEN  -- validate WPI parameter
        SELECT 'Error: Value must be greater than 0 and less than 1' AS RESULT;
    ELSE
        UPDATE PRODUCT
        SET P_DISCOUNT = P_DISCOUNT + WPI
        WHERE P_QOH >= P_MIN * 2;
        SELECT '** Update finished.';
    END IF;
END;
$$
delimiter ;

START TRANSACTION;
call PRC_PROD_DISCOUNT(.33);
SELECT P_CODE, P_DESCRIPT, P_QOH, P_MIN, P_DISCOUNT FROM PRODUCT;
ROLLBACK;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

P_CODE	P_DESCRIPT	P_QOH	P_MIN	P_DISCOUNT
11QER/31	Power painter, 15 psi., 3-nozzle	8	5	0.00
13-Q2/P2	7.25-in. pwr. saw blade	32	15	0.38
14-Q1/L3	9.00-in. pwr. saw blade	18	12	0.00
1546-QQ2	Hrd. doth, 1/4-in., 2x50	15	8	0.00
1558-QW1	Hrd. doth, 1/2-in., 3x50	23	5	0.33
2232/QTY	B&D jigsaw, 12-in. blade	8	5	0.05
2232/QWE	B&D jigsaw, 8-in. blade	6	5	0.05
2238/QPD	B&D cordless drill, 1/2-in.	12	5	0.38
23109-HB	Claw hammer	23	10	0.43
23114-AA	Sledge hammer, 12 lb.	8	5	0.05
54778-2T	Rat-tail file, 1/8-in. fine	43	20	0.33
89-WRE-Q	Hicut chain saw, 16 in.	11	5	0.38
PVC23DRT	PVC pipe, 3.5-in., 8-ft	188	75	0.33
SM-18277	1.25-in. metal screw, 25	172	75	0.33
SW-23116	2.5-in. wd. screw, 50	237	100	0.33
WR3/TT3	Steel matting, 4'x8'x1/6", .5" m...	18	5	0.43

8-7g Triggers

- **Trigger** A procedural SQL code that is automatically invoked by the relational database management system when a data manipulation event occurs.
- A trigger is invoked before or after a data row is inserted, updated, or deleted.
- A trigger is associated with a database table.
- Each database table may have one or more triggers.
- A trigger is executed as part of the transaction that triggered it

8-7g Triggers (cont.)

- For example, if a product's quantity on hand is updated when the product is sold, the system should automatically check whether the quantity on hand falls below its minimum allowable quantity.
- To demonstrate that process, use the PRODUCT table in Figure 8.22. Note the use of the minimum quantity (P_MIN) and product reorder flag (P_REORDER) columns.
- The P_MIN indicates the minimum quantity on hand allowable before restocking an item.
- The P_REORDER column is a numeric field that indicates whether the product needs to be reordered (1 – Yes, – 5 No).
- The initial P_REORDER values are set to 0 (No) to serve as the basis for the initial trigger development.


```
CREATE TRIGGER trigger_name
[BEFORE / AFTER] [DELETE / INSERT / UPDATE] ON table_name
FOR EACH ROW
[DECLARE]
[variable_namedata type[=initial_value] ]
BEGIN
Procedural SQL instructions;
…
END;
```

Review Questions

1. What type of integrity is enforced when a primary key is declared?
2. Explain why it might be more appropriate to declare an attribute that contains only digits as a character data type instead of a numeric data type.
3. What is the difference between a column constraint and a table constraint?
4. What are "referential constraint actions"?
5. What is the purpose of a CHECK constraint?
6. Explain when an ALTER TABLE command might be needed.
7. What is the difference between an INSERT command and an UPDATE command?
8. What is the difference between using a subquery with a CREATE TABLE command and using a subquery with an INSERT command?
9. What is a sequence? Write its syntax.
10. What is a trigger, and what is its purpose? Give an example.
11. What is a stored procedure, and why is it particularly useful? Give an example.

Problems

The Ch08_ConstructCo database stores data for a consulting company that tracks all charges to projects. The charges are based on the hours each employee works on each project. The structure and contents of the Ch08_ConstructCo database are shown in Figure P8.1.

Figure P8.1 The Ch08_ConstructCo Database

Database name: Ch08_ConstructCo

Relational diagram

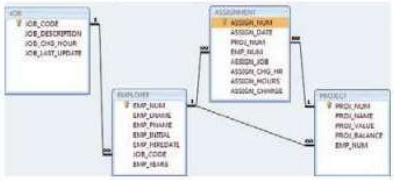


Table name: EMPLOYEE

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE	EMP_YEARS
101	News	John	G	08-Nov-04	502	17
102	Senior	David	H	12-Jul-93	501	28
103	Arbough	June	E	01-Dec-00	500	21
104	Ramoras	Anne	K	15-Nov-91	501	30
105	Johnson	Alice	K	01-Feb-97	502	25
106	Smithfield	William		22-Jun-08	500	13
107	Alonzo	Maria	D	10-Oct-97	500	24
108	Washington	Ralph	B	22-Aug-95	501	26
109	Smith	Larry	W	18-Jul-01	501	20
110	Olenko	Gerald	A	11-Dec-99	505	22
111	Wabash	Geoff	B	04-Apr-95	506	27
112	Smithson	Darlene	M	23-Oct-98	507	23
113	Joenbrood	Delbert	K	15-Nov-00	508	21
114	Jones	Annelise		20-Aug-97	508	24
115	Bawangi	Travis	B	25-Jan-96	501	26
116	Pratt	Gerald	L	05-Mar-01	510	21
117	Williamson	Angie	H	19-Jun-00	509	21
118	Frommer	James	J	04-Jan-09	510	13

Table name: JOB

JOB_CODE	JOB_DESCRIPTION	JOB_CHG_HOUR	JOB_LAST_UPDATE
500	Programmer	35.75	20-Nov-21
501	Systems Analyst	96.75	20-Nov-21
502	Database Designer	123.00	24-Mar-22
503	Electrical Engineer	84.50	20-Nov-21
504	Mechanical Engineer	67.90	20-Nov-21
505	Civil Engineer	55.78	20-Nov-21
506	Clerical Support	26.87	20-Nov-21
507	DSS Analyst	45.95	20-Nov-21
508	Applications Designer	48.10	24-Mar-22
509	Bio Technician	34.55	20-Nov-21
510	General Support	18.36	20-Nov-21

Table name: ASSIGNMENT

ASSIGN_NUM	ASSIGN_DATE	PROJ_NUM	EMP_NUM	ASSIGN_JOB	ASSIGN_CHG_HR	ASSIGN_HOURS	ASSIGN_CHARGE
1001	22-Mar-22	18	103	503	84.5	3.5	295.75
1002	22-Mar-22	22	117	509	34.55	4.2	145.11
1003	22-Mar-22	18	117	509	34.55	2	69.1
1004	23-Mar-22	18	103	503	84.5	5.9	498.55
1005	22-Mar-22	25	108	501	96.75	2.2	212.85
1006	22-Mar-22	22	104	501	96.75	4.2	406.35
1007	22-Mar-22	25	113	508	50.75	3.8	192.85
1008	22-Mar-22	18	103	503	84.5	0.9	76.05
1009	23-Mar-22	15	115	501	96.75	5.6	541.8
1010	23-Mar-22	13	117	509	34.55	2.4	82.92
1011	23-Mar-22	25	105	502	105	4.3	451.5
1012	23-Mar-22	18	108	501	96.75	3.4	328.95
1013	23-Mar-22	25	115	501	96.75	2	193.5
1014	23-Mar-22	22	104	501	96.75	2.8	270.9
1015	23-Mar-22	15	103	503	84.5	6.1	515.45
1016	25-Mar-22	22	105	502	105	4.7	493.5
1017	23-Mar-22	18	117	509	34.55	3.8	131.29
1018	23-Mar-22	25	117	509	34.55	2.2	76.01
1019	24-Mar-22	25	104	501	110.5	4.9	541.45
1020	24-Mar-22	15	101	502	125	3.1	387.5
1021	24-Mar-22	22	108	501	110.5	2.7	298.35
1022	24-Mar-22	22	113	501	110.5	4.9	541.45
1023	24-Mar-22	22	105	502	125	3.5	437.5
1024	24-Mar-22	15	103	503	84.5	3.3	278.85
1025	24-Mar-22	18	117	509	34.55	4.2	145.11

Table name: PROJECT

PROJ_NUM	PROJ_NAME	PROJ_VALUE	PROJ_BALANCE	EMP_NUM
15	Evergreen	1453500	1002350	103
18	Amber Wave	3500500	2110346	108
22	Rolling Tide	805000	500345.2	102
25	Starflight	2650500	2309880	107

Note that the ASSIGNMENT table in Figure P8.1 stores the JOB_CHG_HOUR values as an attribute (ASSIGN_CHG_HR) to maintain historical accuracy of the data. The JOB_CHG_HOUR values are likely to change over time. In fact, a JOB_CHG_HOUR change will be reflected in the ASSIGNMENT table. Naturally, the employee primary job assignment might also change, so the ASSIGN_JOB is also stored. Because those attributes are required to maintain the historical accuracy of the data, they are *not* redundant.

Given the structure and contents of the Ch08_ConstructCo database shown in Figure P8.1, use SQL commands to answer Problems 1–15.

1. Write the SQL code that will create only the table structure for a table named EMP_1. This table will be a subset of the EMPLOYEE table. The basic EMP_1 table structure is summarized in the following table. Use EMP_NUM as the primary key. Note that the JOB_CODE is the FK to JOB so be certain to enforce referential integrity. Your code should also prevent null entries in EMP_LNAME and EMP_FNAME.

Table P8.1 ConstructCo database structure

Attribute (Field) Name	Data Declaration
EMP_NUM	VARCHAR(3)
EMP_LNAME	VARCHAR(15)
EMP_FNAME	VARCHAR(15)
EMP_INITIAL	CHAR(1)
EMP_HIREDATE	DATE
JOB_CODE	VARCHAR(3)

2. Having created the table structure in Problem 1, write the SQL code to enter the first two rows for the table shown in Figure P8.2. Each row should be inserted individually, without using a subquery. Insert the rows in the order that they are listed in the figure.

Figure P8.2 The Contents of the EMP_1 Table

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE
101	News	John	G	08-Nov-04	502
102	Senior	David	H	12-Jul-93	501
103	Arbough	June	E	01-Dec-00	500
104	Ramoras	Anne	K	15-Nov-91	501
105	Johnson	Alice	K	01-Feb-97	502
107	Alonzo	Maria	D	10-Oct-97	501
108	Washington	Ralph	B	22-Aug-95	501
109	Smith	Larry	W	18-Jul-01	501

3. Using the EMPLOYEE table that already exists, use a subquery to insert the remaining rows from the EMPLOYEE table into the EMP_1 table. Remember, your subquery should only retrieve the columns needed for the EMP_1 table and only the employees shown in the figure.
4. Write the SQL code that will save the changes made to the EMP_1 table (if supported by your DBMS).
5. Write the SQL code to change the job code to 501 for the person whose employee number (EMP_NUM) is 107.
6. Write the SQL code to delete the row for William Smithfield, who was hired on June 22, 2008, and whose job code is 500. (*Hint:* Use logical operators to include all of the information given in this problem. Remember, if you are using MySQL, you will have to first disable "safe mode.")
7. Write the SQL code to create a copy of EMP_1, including all of its data, and naming the copy EMP_2.

8. Using the EMP_2 table, write the SQL code that will add the attributes EMP_PCT and PROJ_NUM to EMP_2. The EMP_PCT is the bonus percentage to be paid to each employee. The new attribute characteristics are:

   EMP_PCT DECIMAL(4,2)
   PROJ_NUM CHAR(3)

9. Using the EMP_2 table, write the SQL code to change the EMP_PCT value to 3.85 for the person whose employee number (EMP_NUM) is 103.
10. Using the EMP_2 table, write a single SQL command to change the EMP_PCT value to 5.00 for the people with employee numbers 101, 105, and 107.
11. Using the EMP_2 table, write a single SQL command to change the EMP_PCT value to 10.00 for all employees who do not currently have a value for EMP_PCT.
12. Using the EMP_2 table, write the SQL command to add .15 to the EMP_PCT of the employee whose name is Maria D. Alonzo. (Use the employee name in your command to determine the correct employee.)
13. Using a single command sequence with the EMP_2 table, write the SQL code that will change the project number (PROJ_NUM) to 18 for all employees whose job classification (JOB_CODE) is 500.
14. Using a single command sequence with the EMP_2 table, write the SQL code that will change the project number (PROJ_NUM) to 25 for all employees whose job classification (JOB_CODE) is 502 or higher.
15. Write the SQL code that will change the PROJ_NUM to 14 for employees who were hired before January 1, 1998, and whose job code is at least 501. When you finish Problems 7–15, the EMP_2 table will contain the data shown in Figure P8.15.

Figure P8.15 The EMP_2 Table Contents

EMP_NUM	EMP_LNAME	EMP_FNAME	EMP_INITIAL	EMP_HIREDATE	JOB_CODE	EMP_PCT	PROJ_NUM
101	News	John	G	08-Nov-00	502	5	25
102	Senior	David	H	12-Jul-89	501	10	14
103	Arbough	June	E	01-Dec-96	500	3.85	18
104	Ramoras	Anne	K	15-Nov-87	501	10	14
105	Johnson	Alice	K	01-Feb-93	502	5	14
107	Alonzo	Maria	D	10-Oct-93	501	5.15	14
108	Washington	Ralph	B	22-Aug-91	501	10	14
109	Smith	Larry	W	18-Jul-97	501	10	

Use the database tables in Figure P8.16 as the basis for Problems 16–30.

Figure P8.16 Ch08_SimpleCo Database Tables

Database name: Ch08_SimpleCo

Table name: CUSTOMER

CUST_NUM	CUST_LNAME	CUST_FNAME	CUST_BALANCE
1000	Smith	Jeanne	1050.11
1001	Ortega	Juan	840.92

Table name: INVOICE

INV_NUM	CUST_NUM	INV_DATE	INV_AMOUNT
8000	1000	23-Mar-22	235.89
8001	1001	23-Mar-22	312.82
8002	1001	30-Mar-22	528.10
8003	1000	12-Apr-22	194.78
8004	1000	23-Apr-22	619.44

TRƯỜNG ĐẠI HỌC KINH TẾ QUỐC DÂN
KHOA TOÁN KINH TẾ

16. Create the CUSTOMER table structure illustrated in Figure P8.16. The customer number should store integer values. The name attributes should support variable length character data up to 30 characters each. The customer balance should support up to six digits on the left of the decimal place and two digits to the right of the decimal place.
17. Create the INVOICE table structure illustrated in Figure P8.16. The invoice number should store integer values. The invoice date should store date values. The invoice amount should support up to 8 digits to the left of the decimal place and two digits to the right of the decimal place.
18. Write the set of SQL commands necessary to insert the data into the CUSTOMER table you created in Problem 16, as illustrated in Figure P8.16.
19. Write the set of SQL commands necessary to insert the data into the INVOICE table you created in Problem 17, as illustrated in Figure P8.16.
20. Enable auto increment to generate values for customer numbers. The values should start with 2000.
21. Enable auto increment to generate values for invoice numbers. The values should start with the value 9000.
22. Insert the following customer into the CUSTOMER table, using the auto increment from Problem 20 to generate the customer number automatically:

    'Powers', 'Ruth', 500
23. Modify the CUSTOMER table to include the customer's date of birth (CUST_DOB), which should store date data.
24. Modify customer 1000 to indicate the date of birth as March 15, 1989.
25. Modify customer 1001 to indicate the date of birth as December 22, 1988.

26. Create a trigger named **trg_updatecustbalance** to update the CUST_BALANCE in the CUSTOMER table when a new invoice record is entered. (Assume that the sale is a credit sale.) Whatever value appears in the INV_AMOUNT column of the new invoice should be added to the customer's balance. Test the trigger using the following new INVOICE record, which would add 225.40 to the balance of customer 1001:

    8005, 1001, '2022-04-27', 225.40

27. Write a procedure named **prc_cust_add** to add a new customer to the CUSTOMER table. Use the following values in the new record:

    1002, 'Rauthor', 'Peter', 0.00

    (You should execute the procedure and verify that the new customer was added to ensure your code is correct.)

28. Write a procedure named **prc_invoice_add** to add a new invoice record to the INVOICE table. Use the following values in the new record:

    8006, 1000, '2022-04-30', 301.72

    (You should execute the procedure and verify that the new invoice was added to ensure your code is correct.)

29. Write a trigger to update the customer balance when an invoice is deleted. Name the trigger **trg_updatecustbalance2**.

30. Write a procedure to delete an invoice, giving the invoice number as a parameter. Name the procedure **prc_inv_delete**. Test the procedure by deleting invoices 8005 and 8006.

Use the Ch08_SaleCo2 database shown in Figure P8.31 to work Problems 31–34.

Figure P8.31 Ch08_SaleCo2 Database Tables

Database name: Ch08_SaleCo2

Table name: CUSTOMER

CUS_CODE	CUS_LNAME	CUS_FNAME	CUS_INITIAL	CUS_AREACODE	CUS_PHONE	CUS_BALANCE
10010	Ramas	Alfred	A	615	844-2573	0.00
10011	Dunne	Leona	K	713	894-1238	0.00
10012	Smith	Kathy	W	615	894-2285	345.86
10013	Olowski	Paul	F	615	894-2180	536.75
10014	Orlando	Myron		615	222-1672	0.00
10015	O'Brian	Amy	B	713	442-3381	0.00
10016	Brown	James	G	615	297-1228	221.19
10017	Williams	George		615	290-2556	768.93
10018	Farriss	Anne	G	713	382-7185	216.55
10019	Smith	Olette	K	615	297-3809	0.00

Table name: INVOICE

INV_NUMBER	CUS_CODE	INV_DATE	INV_SUBTOTAL	INV_TAX	INV_TOTAL
1001	10014	16-Jan-22	24.90	1.99	26.89
1002	10011	16-Jan-22	9.98	0.80	10.78
1003	10012	16-Jan-22	153.85	12.31	166.16
1004	10011	17-Jan-22	34.97	2.80	37.77
1005	10018	17-Jan-22	70.44	5.64	76.08
1006	10014	17-Jan-22	397.83	31.83	429.66
1007	10015	17-Jan-22	34.97	2.80	37.77
1008	10011	17-Jan-22	399.15	31.93	431.08

Table name: PRODUCT

P_CODE	P_DESCRIPT	P_INDATE	P_QOH	P_MIN	P_PRICE	P_DISCOUNT	V_CODE
11QER/31	Power painter, 15 psi., 3-nozzle	03-Nov-21	8	5	109.99	0.00	25595
13-Q2/P2	7.25-in. pwr. saw blade	13-Dec-21	32	15	14.99	0.05	21344
14-Q1/L3	9.00-in. pwr. saw blade	13-Nov-21	18	12	17.49	0.00	21344
1546-QQ2	Hrd. cloth, 1/4-in., 2x50	15-Jan-22	15	8	39.95	0.00	23110
1558-QW1	Hrd. cloth, 1/2-in., 3x50	15-Jan-22	23	5	43.99	0.00	23115
2232/QTY	B&D jigsaw, 12-in. blade	30-Dec-21	8	5	109.92	0.05	24288
2232/QWE	B&D jigsaw, 8-in. blade	24-Dec-21	6	5	99.87	0.05	24288
2238/QPD	B&D cordless drill, 1/2-in.	20-Jan-22	12	5	38.95	0.05	25595
23109-HB	Claw hammer	20-Jan-22	23	10	9.95	0.10	21225
23114-AA	Sledge hammer, 12 lb.	02-Jan-22	8	5	14.40	0.05	
54778-2T	Rat-tail file, 1/8-in. fine	15-Dec-21	43	20	4.99	0.00	21344
89-WRE-Q	Hicut chain saw, 16 in.	07-Feb-22	11	5	256.99	0.05	24288
PVC23DRT	PVC pipe, 3.5-in., 8-ft	20-Feb-22	188	75	5.87	0.00	
SM-18277	1.25-in. metal screw, 25	01-Mar-22	172	75	6.99	0.00	21225
SW-23116	2.5-in. wd. screw, 50	24-Feb-22	237	100	8.45	0.00	21231
WR3/TT3	Steel matting, 4'x8'x1/6", .5" mesh	17-Jan-22	18	5	119.95	0.10	25595

Table name: LINE

INV_NUMBER	LINE_NUMBER	P_CODE	LINE_UNITS	LINE_PRICE	LINE_TOTAL
1001	1	13-Q2/P2	1	14.99	14.99
1001	2	23109-HB	1	9.95	9.95
1002	1	54778-2T	2	4.99	9.98
1003	1	2238/QPD	1	38.95	38.95
1003	2	1546-QQ2	1	39.95	39.95
1003	3	13-Q2/P2	5	14.99	74.95
1004	1	54778-2T	3	4.99	14.97
1004	2	23109-HB	2	9.95	19.90
1005	1	PVC23DRT	12	5.87	70.44
1006	1	SM-18277	3	6.99	20.97
1006	2	2232/QTY	1	109.92	109.92
1006	3	23109-HB	1	9.95	9.95
1006	4	89-WRE-Q	1	256.99	256.99
1007	1	13-Q2/P2	2	14.99	29.98
1007	2	54778-2T	1	4.99	4.99
1008	1	PVC23DRT	5	5.87	29.35
1008	2	WR3/TT3	3	119.95	359.85
1008	3	23109-HB	1	9.95	9.95

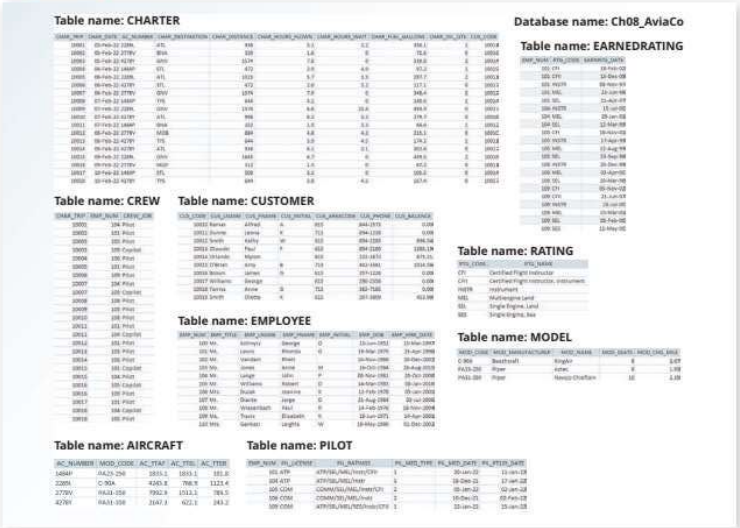
Table name: VENDOR

V_CODE	V_NAME	V_CONTACT	V_AREACODE	V_PHONE	V_STATE	V_ORDER
21225	Bryson, Inc.	Smithson	615	223-3234	TN	Y
21226	SuperLoo, Inc.	Flushing	904	215-8995	FL	N
21231	D&E Supply	Singh	615	228-3245	TN	Y
21344	Gomez Bros.	Ortega	615	889-2546	KY	N
22567	Dome Supply	Smith	901	678-1419	GA	N
23119	Randsets Ltd.	Anderson	901	678-3998	GA	Y
24004	Brackman Bros.	Browning	615	228-1410	TN	N
24288	ORDVA, Inc.	Hakford	615	898-1234	TN	Y
25443	B&K, Inc.	Smith	904	227-0093	FL	N
25501	Damal Supplies	Smythe	615	890-3529	TN	N
25595	Rubicon Systems	Orton	904	456-0092	FL	Y

31. Create a trigger named **trg_line_total** to write the LINE_TOTAL value in the LINE table every time you add a new LINE row. (The LINE_TOTAL value is the product of the LINE_UNITS and LINE_PRICE values.)
32. Create a trigger named **trg_line_prod** that automatically updates the quantity on hand for each product sold after a new LINE row is added.
33. Create a stored procedure named **prc_inv_amounts** to update the INV_SUBTOTAL, INV_TAX, and INV_TOTAL. The procedure takes the invoice number as a parameter. The INV_SUBTOTAL is the sum of the LINE_TOTAL amounts for the invoice, the INV_TAX is the product of the INV_SUBTOTAL and the tax rate (8 percent), and the INV_TOTAL is the sum of the INV_SUBTOTAL and the INV_TAX.
34. Create a procedure named **prc_cus_balance_update** that will take the invoice number as a parameter and update the customer balance. (*Hint*: You can use DECLARE to define a TOTINV numeric variable that holds the computed invoice total.)

Use the Ch08_AviaCo database shown in Figure P8.35 to work Problems 35–46.

Figure P8.35 Ch08_AviaCo Database Tables



35. Modify the MODEL table to add the attribute and insert the values shown in the following table.

Table P8.35 Attribute and Values for Problem 35

Attribute Name	Attribute Description	Attribute Type	Attribute Values
MOD_WAIT_CHG	Waiting charge per hour for each model	Numeric	$100 for C-90A $50 for PA23-250 $75 for PA31-350

36. Write the queries to update the MOD_WAIT_CHG attribute values based on Problem 35.
37. Modify the CHARTER table to add the attributes shown in the following table.

Table P8.37 Attributes and Values for Problem 37

Attribute Name	Attribute Description	Attribute Type
CHAR_WAIT_CHG	Waiting charge for each model (copied from the MODEL table)	Numeric
CHAR_FLT_CHG_HR	Flight charge per mile for each model (copied from the MODEL table using the MOD_CHG_MILE attribute)	Numeric
CHAR_FLT_CHG	Flight charge (calculated by CHAR_HOURS_FLOWN × CHAR_FLT_CHG_HR)	Numeric
CHAR_TAX_CHG	CHAR_FLT_CHG × tax rate (8%)	Numeric
CHAR_TOT_CHG	CHAR_FLT_CHG + CHAR_TAX_CHG	Numeric
CHAR_PYMT	Amount paid by customer	Numeric
CHAR_BALANCE	Balance remaining after payment	Numeric

38. Write the sequence of commands required to update the CHAR_WAIT_CHG attribute values in the CHARTER table. (*Hint*: This could be done with an update, an updatable view, or a stored procedure.)
39. Write the sequence of commands required to update the CHAR_FLT_CHG_HR attribute values in the CHARTER table.
40. Write the command required to update the CHAR_FLT_CHG attribute values in the CHARTER table.
41. Write the command required to update the CHAR_TAX_CHG attribute values in the CHARTER table.
42. Write the command required to update the CHAR_TOT_CHG attribute values in the CHARTER table.
43. Modify the PILOT table to add the attribute shown in the following table.

Table P8.43 Attribute to Add in Problem 43

Attribute Name	Attribute Description	Attribute Type
PIL_PIC_HRS	Pilot in command (PIC) hours; updated by adding the CHARTER table's CHAR_HOURS_FLOWN to the PIL_PIC_HRS when the CREW table shows the CREW_JOB to be Pilot	Numeric

44. Create a trigger named **trg_char_hours** that automatically updates the AIRCRAFT table when a new CHARTER row is added. Use the CHARTER table's CHAR_HOURS_FLOWN to update the AIRCRAFT table's AC_TTAF, AC_TTEL, and AC_TTER values.

45. Create a trigger named **trg_pic_hours** that automatically updates the PILOT table when a new CREW row is added and the CREW table uses a Pilot CREW_JOB entry. Use the CHARTER table's CHAR_HOURS_FLOWN to update the PILOT table's PIL_PIC_HRS only when the CREW table uses a Pilot CREW_JOB entry.
46. Create a trigger named **trg_cust_balance** that automatically updates the CUSTOMER table's CUS_BALANCE when a new CHARTER row is added. Use the CHARTER table's CHAR_TOT_CHG as the update source. (Assume that all charter charges are charged to the customer balance.)